

Concepts:

Database: Its used to store structured data only. Its mainly for OLTP and supports SQL queries only, not for big data analysis

Data Warehouse: Its used for Analytical process/querying with huge past data (OLAP), its only for structured data. It needs resources(CPU, RAM, Disk(storage) and can't easily scalable. Schema mismatch not accepted. Ex: Redshift, PostgreSQL...etc

Data lake: It resolves limitations of traditional data warehouse by supporting all types of data with unlimited storage capacity, highly scalable. But it poor support DML(SQL) queries and during some transformations it may lead to data corruption. Here even Schema mismatch data files will be allowed to storage.

Ex: S3, GCS, ADLS...etc

Delta lake: It's a open format storage layer for all types of data (un-structure, semi-structured, structured) and streaming also and with reliability, security, high scalability and high performance for both batch & streaming and heart of Lake House (Data Lake+Data warehouse). and its supports ACID properties of SQL. Here also even schema mismatch data files will be allowed to storage. It additionally stores metadata in form of Delta log folder(meta data) along with parquet files (actual data).

Delta Lake house: It's a

Delta lake Architecture: It's during DML queries in delta tables, transaction logs created in Json format, with these we can do time travel for rollbacks. For each 10 Json log files, 1 parquet checkpoint will be created for simplifying the process.

CRC(cyclic redundant check) file to prevent accidental damage of data in delta lake table. In this, it follows soft delete policy to remove log files when we run delete query, by default 7 days retain period, we can config it up to 30 days also

OLTP (Online Transaction Processing) : Handles transaction data with fast responses. (Current daily transaxctions)

OLAP (Online Analytical Processing): Supports analyzing large datasets with flexible queries mainly on past 10 yrs data...etc

Datalake: Its scalable, cost-effective for storing diversified data types(structured/semi-structured/UN-structured)

ADLS Gen2: Built on Az Blob storage with hierarchical Name-spaces for better data organizing & processing. Allows Agile ELT workflows, enable quick data explore. Supports Delta Formats for ACID properties & efficient transformations with ADB

Storage Redundancy:

LRS: Stores data within a single Region

ZRS: Distributes data across zones for better resilience.

GRS: Copies data to paired Regions, useful in regional failures

Access Tiers:

Hot Tier: Frequently accessing data, Immediate access

Cool Tier: Low cost, suited for in-frequent accessing data

Cold Tier: cost-effective for long-term storage (backups)

Archive Tier: Cheapest for rarely accessing data, with longer retrieval times

Life Cycle Management: To automate Tier Transitions and to balance cost & performance by monitor usage patterns.

CSV: Simple, but requires careful validation due to schema limitations

XML: Flexible & Robust for complex data structures but less efficient.

Json: widely used for APIs, light weight, easy to process

Parquet: A Modern binary file format designed for analytical workloads. Superior data retrieval speed, compression, better columnar storage efficiency and faster query by accessing the only required columns.

Delta Lake: It enhances data lakes by adding extra features ACID, Schema evolution, historical data tracking(version,Time Travel) Robust for large-scale data processing. Built-on as parquet files with Json Transaction log for reliability. Batch & Streaming Selective updates by modify only needed files. Rollbacks possible. Supports Parallelism(distribute data across multi-nodes) Partitioning(Organizing data into small partitions for fast querying), Data Integrity (No conflicts during concurrent Tasks). Durability(Safeguard committed transactions against system failures). helps best audits by tracks detailed data logs. Supports Vacuum operation to remove unused data files to optimize storage. Supports Merge command for schema evolution, implements checks to avoid corruption, prevents unexpected schema changes(schema enforcement). Unified Process for Batch & Stream. Optimize(combine samll files into single large file),CustomOrdering(efficient retrieval)

Data Pipeline: Its an Automated workflow to move data from one place to another place by scheduled or Triggered manner.

Database: It's a digital storage space for organized data which we can easily access, search, update...etc by querying it for info.

Schema: It's a blueprint/design how data is organized and related i.e. how tables are connected in a DB.

Table: It's a rows and columns based data storing option.

ETL: Extract(collect data from diff sources) --> Transform(clean/modify data) --> Load(save transformed data into DB/DWH/ADLS)

ELT: Extract(collect data from diff sources) --> Load(save data into DB/DWH/ADLS) --> Transform(clean/modify data)

Data Lake: Its centralized repo for storing structured/semi-structured/unstructured data with scalability feature.

Data warehouse: A central place where cleaned/transformed data is stored for Analysis. Company's main data archive.

Data Mart: A smaller section of DWH focused on a specific team/purpose for one dept.

Batch Processing: Processing a large amount of data all at once as chunks/batches on a scheduled timings.

Stream Processing: Handling data as it arrives immediately processing as live stream instead of downloading

Data Quality: Measuring how cleaned, complete, accurate our data is to make sure its reliable & useful. (by Filter Null/duplicates)

Data Modeling: Designing a visual Map/chart of data to show how diff pieces relate to each other. Star & Snowflake schemas

Data Orchestration: Automating and Coordinating how data moves b/w systems.

Data Lineage: Tracking the journey of data from source --> sink, where it comes, what happens to it and where it ends up.

Data Lake house: A Hybrid system combining raw data storage of data Lake(ADLS) with structured organization of DWH for unified Analytics. Here delta lake concept is key role means it acts as abstraction layer of metadata on datalake

Data Partitioning: Split the large datasets into smaller partitions to improve query performance and manageability.
Ex: Partitioning by year/Month/day (or) by Country (or) by dept...etc

Data Sharding: Breaking down large datasets and distribute horizontally across multiple DB instances to improve scalability.

Schema Evolution: Adapting ability to changes in structure of data source and its schema (adding new columns in a Table)

Change Data Capture(CDC): Capture & Tracking changes in source data for real-time updates.
Ex: Using Debezium to Track changes in MySQL DB and load it into Apache Kafka Topic

Data Governance: Defining Rules, Policies, standards to manage & access data to whom(users) & at what levels(Roles).

Data Security: Protect/Encrypt data at rest, in transit and ensure secure access by authenticated ways. (by Hash Algorithms)
Ex: Azure Key Vault secrets, Databricks Secret Scopes

Data Anonymization: Maintain secrecy by Mask with personal identifying info (User IDs, Phone number/emails...etc) before sending to other Analysis Teams.

Metadata Management: Store & manage info about the data (source, structure and usage) Ex: Hive & Unity Meta-stores

Columnar Storage: Store data in columnar format for quick analysis, better performance by limiting needed data scans only

Data Normalization: Organize data to reduce redundancy(duplicacy) and improve integrity. Structuring client data in 3NF schema

No-SQL DBs: Non-relational DBs to store semi-structured data. Ex: MongoDB to store data as Json documents type

Distributed Systems: Systems that divide the Tasks across multiple Nodes for quick processing by using parallelism (HDFS, Spark)

Data Replication: Creating redundant copies of data for future fail-over & maintain high availability(LRS/ZRS/GRS)

Data Backup: Creating backup copies of data for disaster recovery purposes.

Load Balancing: Distribute data processing workloads across multiple nodes to avoid burdening on single node.

Workflow Orchestration: Manage dependencies, schedules and Monitoring data pipeline workflows. Ex: Apache Airflow DAGs

Data Validation: Testing data for expected values/types before processing into production env. (by python data validation scripts)

Data Transformation: Converting raw data into cleaned usable format by aggregations, joins make ready for business Analytics

OLAP & OLTP: OLAP for historical Analytics, OLTP for day-to-day transaction processing (real-time purchases/Banking...etc)

SQL Query Optimization: Adding Indexes, rewriting Joins can improve queries for better performance & fast execution.

Serverless Data Processing: Processing data without managing underlying servers. (Synapse built-in serverless)

Event-driven processing: Triggering data processing workflows against Events happening. (New uploads into Blob storage...etc)

Job Scheduling: Automating the execution of jobs at specific Intervals (ETL jobs at ADF Schedule Triggers, databricks Job schedule)

Data Schema Registry: Storing Schema definitions for data to facilitate consistent data exchange.

Data versioning: keeping track of changes applied with data and datasets for future restores. (Versions at delta tables)

Data Cataloging: Indexing data assets to make them searchable and discoverable (by Azure Purview service)

Data ingestion: Pulling in data from different data sources for storage/processing. (API/github/on-prem --> ADLS/AzureSQL)

Data Lake Storage: Its for flexible data storing facility of any kind of data at scale.

Data Cleansing: Correcting/removing the corrupted/incomplete/inaccurate data (Ex: Remove Nulls)

Master data Management: Maintain Unified view of customer data by Informatica MDM (single, trusted source of business data)

Cloud Data Services: Leverage/Adapt cloud services Technology for data storage/processing.

Azure Synapse :

Limitless Analytics service and provides many services together [Integration(ADF), Enterprise Dataware housing, Bigdata Capabilities, ADLS, Databricks Notebooks, SQL, Pyspark and PowerBI...etc] and its based on Massive Parallel Processing (MPP) Architecture i.e Control Node --> Compute Node (Distributed Data) permits Max 60 distributions only

SQL pools: Group of computational resources which use compute infra in a shared basis. [1. Serverless 2. Dedicated]

In Dedicated SQL pools, compute infra config is under our control, whereas in serverless(built-in) pool Azure managed all. DWU is measurement of dedicated SQL pool compute infra [DWU = CPU + Memory + i/o]

Control Node hosts the distributed query engine (MPP engine) optimizes the queries by parallel processing. Compute Nodes store all the data in ADLS and run the queries in parallel. DMS in compute Node is helped for data movement across the Compute Nodes for parallel queries & outputs.

Task will be divided into 60 smaller queries and each query runs on each data distribution. Storage & Compute costs both applicable and Except pausing state, remaining time compute is much costly even we not use for queries. It is scalable. But in server-less(built-in) pool, only storage cost, No compute cost and also here without loading data into synapse tables, we can query ADLS data directly & Auto-Scale. But in ADF, we need to load from ADLS into SQL DB by copy activity.

Data Distribution Types: It determines how data is distributed & stored onto tables getting ready for queries.

1. Hash distributed: It is highest query performance while using Joins & Aggregations on Huge sized Tables. Hash Function used to assign rows to distributions. Data will gone to same distribution which having same hash value.
2. Round Robin: It gives fast performance while staging table used during loads. It evenly distribute data across the table, but no optimizations, it assigns rows sequential to distributions. For heavy Joins, reshuffle takes place with delay.
3. Replicated: It is Faster for smaller tables only. It copies full copy of table to each compute node. Here more storage will be wasted due to multiple replicas.

Synapse --> Create [RG, Workspace Name, Region, ADLS Name, FileSystem Name --> Review+Create
User Name, password to be given for SQL pool [Ex: admin, Azure@123]

Synapse --> Data --> Linked (ADLS) --> right click on needed csv/json/parquet files and select the query, output will come
No need of ADF copy ETL pipeline (ADLS --> SQL DB) for this.

Azure Devops:

Its fully Managed cloud-based service that automates the process of building, testing, deploying code. With powerful features like Yaml-based config, automated testing and flexible deployment options, Devops practices are to deliver applications quickly and with high quality. By using Yaml pipelines development teams can automate build, test and deployment processes efficiently.

Yaml pipeline is defined by several key sections that represent the stages, jobs and tasks in CI/CD process. Each section helps manage different steps like building, testing and deploying the application. Main components of pipeline are as follows:

Trigger, Pool, Stages and Jobs, Variables, Artifacts, deployment

Benefits/Features of Azure Devops Yaml pipelines: Version Control, Flexibility, Re-usability, Portability

To Create Release pipeline, by pre-built Templates (Azure App Service deployment, Kubernetes deployment) or start with an empty job. Release pipeline is divided into stages that represent different env [dev, staging, prod]

Link Artifacts: Choose latest version of Artifact (output of CI build pipeline) & configure pipeline

Triggers: Enable Continuous Deployment Trigger to automatically create a release when a new build is available.

Variables: By using variables, we can define different values for each stage (dev API key, prod API key).

Approval Gates: With this, we can control release pipeline by manual approval before deploying to production

Benefits of Azure Devops: Increased Development Speed, Early Bug Detection, Improved Code Quality, Consistency in Deployment, Scalability

Azure Databricks is an Apache Spark-based analytics platform designed for big data processing with interactive workspace notebooks to collaborate on large-scale data transformations.

DB: It is a system which allowing users to store & organize data user wise.

SQL: Structured Query Language for manage & manipulate relational DBs

CRUD: It's a acronym used in SQL (Create, Read, Update, Delete)

SQL Commands: DDL(Create/Drop/Alter/Truncate/Rename), DML(Insert/Update/Delete/Select), DCL(Grant/Revoke), TCL(Commit/RollBack/SavePoint)

Data Type: It defines which type of value can be stored into that specified column

[String(Char,Varchar), Numeric(Int,Float,Boolean), Date & time(date, datetime)]

Primary key: Uniquely identified in each record in a table, cannot Null, cannot duplicate

Foreign Key: It exists in a table & it refers primary key in another table and mainly establishes relation b/w both tables. It can have Null & duplicate values also.

Constraints: These are Rules applied to table columns

Primary key(Uniquely identified each record), **Foreign Key**(Referential integrity), **Unique**(all values distinct),

Check(values of a column satisfies specific condition), **Not Null**(Column should not having Nulls), **default**(specifying default value for entire column), **Create Index** (To create & retrieve data from DB very quickly)

Retrieval query: `select * from employees;`

Delete: Removes rows from table based on a condition and can be rolled back, triggers can be invoked.

Truncate: Removes all rows from table, faster but can't rolled back, triggers can't be invoked.

Max Salary: `select Max(Salary) from employees;`

2nd Highest Salary: `select Max(salary) from employees where salary < (select Max(salary) from employees);`

Join: Combine rows from 2 or more tables based on a related column. [inner, left, right, full, cross]

Fetch Employee & dept Names by Join: `select e.name,d.dept_name from employees e join dept d on e.dept_Id = d.Id;`

Group by: It groups rows with the same values into summary rows. It commonly used with agg functions count, sum, avg...etc

Count employees dept wise: `select dept_id, count(*) from employees group by dept_id;`

Where : Filters rows before grouping (applied to individual rows)

Having: Filters groups after group by clause (applies to agg functions)

Fetch dept's more than 5 employees: `select dept_id, count(*) from employees group by dept_id having count(*) > 5;`

Union: Combines results of 2 or more select queries and removes duplicates.

UnionAll: Combines results but keep all duplicates.

Sub-Query: Query nested within another query, here it retrieves data and passed it to outer query

Employees whose salary greater than Avg Salary: `select * from employees where salary > (select avg(salary) from employees);`

Inner Join: It returns matching valued rows only from both tables

Outer Join: It Returns Matching + Non-Matching Rows from 1 or both tables

Fetch Current Date: `select current_date;`

Indexing: It improves data retrieval speed by providing data structure(index) on 1 or more columns of table.

Normalization: Process of organizing data to reduce redundancy(duplicacy) and improve data integrity

De-Normalization: process of combining normalized tables to improve performance by introducing redundancy

Email Column to be add: `alter table employees add column email varchar(100);`

Stored Procedure: set of sql statements can be stored in DB and executed as a program/script for a specific task.

```
create procedure GetAllEmployees()
begin
    select * from employees;
end;
```

Triggers: These are special procedures are automatically executed response to certain events (insert, delete, update) on a table.

Trigger logs for delete action:

```
create trigger log_delete
after delete on employees
for each row
begin
    insert into log_table(action, emp_id, log_time) values("DELETE", old.id, now());
end;
```

View: It's a virtual table based on result set of a query. It doesn't store the data but provides simplified way for complex queries

View for Employees salary greater than 50000: `Create view nnview1 as select * from employees where salary > 50000;`

View Vs Table: Table stores data physically. But a view is virtual representation, it dynamically pulls data from 1 or more tables without storing data.

Aggregate Functions: Count(), Max(), Min(), Sum(), Avg()

Total Salary dept wise: `select dept_id, sum(salary) from employees group by dept_id;`

Distinct: It returns unique values from a column by eliminate duplicate entries

Distinct Job Titles from Employees: `select distinct job_title from employees;`

ACID: These properties are ensure reliable processing of DB Transactions

Atomicity: Ensures all parts of transaction completed successfully or none at all.

Consistency: Ensures the DB remains in valid state before & after the transaction. No impact of wrong transactions on DB

Isolation: Ensures transactions don't affect each other's execution

Durability: Ensures once a transaction is committed, it remains permanent, even in the event of failure also.

Transaction: It's a sequence of 1 or more SQL operations treated as a single unit of work.

Commit: Saves all changes made during the current transaction

RollBack: Undo the changes made during the current transaction, restore the DB to previous state

SavePoint: Setting a point within a transaction to which we can rollback later.

Ex: `start transaction; update employees set salary = 60000 where id=1; commit;`

Case: Its used to perform conditional logic in SQL queries allowing multiple diff outputs based on conditions

```
Select name,
case
    when salary > 50000 then 'High'
    when salary between 30000 and 50000 then 'Medium'
    else 'low'
end as salary_category
from employees;
```

Null: It represents Absence of a value in DB Table. Its not equal to 0 or empty string.

Ex: `select * from employees where email is Null;`

Coalesce: It returns 1st Not-Null value in a list of expressions by avoiding Nulls in each row.

Ex: `select name,coalesce(email,"No Email") as email_address from employees;`

Count(*) Vs Count(any Column): Count(*) counts all rows in a table including Null Values, but count(any cloumn) counts only Not-Null values in that specified column.

Char Vs Varchar: Char is fixed length string(even string shorter, it occupies specified memory), but Varchar is variable-length string (it uses necessary memory space only from specified length,it saves memory).

Update employee salary by 10% below 30k: `update employees set salary = salary * 1.10 where salary < 30000;`

Recursive query: It references by itself and handles hierarchical data (organizational structures)

```
with recursive employeehierarchy as
    (Select id,name,manager_id from employees where manager_id is Null
    unionall
    Select e.id,e.name,e.manager_id from employees e inner join employeehierarchy eh on e.manager_id = eh.id )
Select * from employeehierarchy;
```

Exists: It checks for existence of rows returned by subquery, it returns True when subquery returns 1 or more rows.

Limit: It specify the max no. of records in output result.

Top 5 highest salaries: `select distinct salary from employees order by salary desc limit 5;`

Composite key: It is combination of 2 or more columns to identify unique records when single column not sufficient for this.

Alter Table : It is used to modify existing table columns (adding/dropping/modifying columns & constraints)

Drop Column: `alter table employees drop column address;`

Data Integrity: It refers accuracy, consistency and reliability of data over its life cycle (by using constraints, validation, DB Rules)

Inner/Left Joins: Inner Join gives only matched rows, Left Join gives All Rows from Left Table with matched rows from right table

Adv of Indexes: It improves query performance by reducing the data to be scanned. It speeds up search, sort, joins, but slows down updates, inserts, deletions by the overhead of maintaining indexes.

Avg Salary each dept: `select dept_id, avg(salary) from employees group by dept_id;`

Cross Join: It produces cartesian product of 2 tables means each row of 1st table combines with each of row 2nd table.

Ex: `select * from employees cross join dept;`

SQL injection: When an attacker can manipulate SQL query by injects malicious inputs through user inputs by unauthorized access to DB, it breaches security, called vulnerability by SQL injection attack.

prevention: Use Parameterized queries, Limit DB user permissions, Using ORM frameworks, validate/sanitize user inputs

DB view: It's a virtual tables based on a query and some views can be update depends on complexity of underlying query.

Rank/Dense_Rank: Rank assigns Ranks within a partition with gaps for Ties, But Dense_Rank assign ranks without gaps for ties.

Ex: `select name,rank() over(order by salary desc) as salary_rank from employees;`

Schema: It is a Logical container for DB objects (Tables/Views/Indexes/Stored Procedures). It defines structure/organization of data.

With Clause(CTE): It defines a Temp result set and can be refer in select/insert/update/delete statements to simplify complex query

CTE query for Employees with salaries > Avg:

```
with CTE as
  (Select avg(salary) as Avg_salary from employees)
  Select * from employees where salary > (Select Avg_salary from CTE);
```

SQL Data Types: Int, Varchar, Date, Float, Boolean

Group by: It groups the same valued rows in a specific columns into aggregated data

Ex: `select dept_id, count(*) as employee_count from employees group by dept_id order by employee_count desc;`

Self-Join: It joins the table with itself only. It used when we need to compare rows within the same table.

Ex: `select e1.name as emp ,e2.name as manager from employees e1 left join employees e2 on e1.manager_id = e2.id;`

Where/And: **Where** is used to filter rows based on specific conditions, But **And** is logical operator to combine multiple conditions.

Ex: `select * from employees where salary between 30000 and 50000;`

Stored Functions: similar to Stored Procedures but returns single value and used in SQL queries as built-in functions

```
create function Annual_Sal(monthly_salary decimal(10,2))
returns decimal(10,2)
begin
  return monthly_salary * 12;
end;
```

Explain: It provides info about how a SQL query will be executed (chosen indexes, join types, estimated rows...etc). It helps for optimized queries & improved performance.

Ex: `explain select * from employees where dept_id = 1;`

Import csv file to SQL DB:

```
create table customer(cust_id int primary key, first_name varchar(50), last_name varchar(50), email varchar(100), address_id int)
copy customer(cust_id, first_name, Last_name, email, address_id) from 'E:\nn-data\customers.csv' delimiter ',' csv header;
```

String Functions: It works on input string and returns output as string

`upper(), lower(), length(), substring(), now(), format(), concat(), replace(), trim()`

TimeStamps, Extract:

```
show timezone --> select now() --> select timeofday() --> select current_time --> select current_date
Select extract(month from doj) from employees
```

Window Functions:

```
Select Marks, st_subject,
sum(Marks) over(partition by st_subject order by Marks) as "Total",
avg(Marks) over(partition by st_subject order by Marks) as "Avg",
count(Marks) over(partition by st_subject order by Marks) as "Count",
min(Marks) over(partition by st_subject order by Marks) as "Min",
max(Marks) over(partition by st_subject order by Marks) as "Max",
from student;
```

Window Functions(Agg function)

```
Select Marks, st_subject,
sum(Marks) over(order by Marks rows between unbounded preceding and unbounded following) as "Total",
avg(Marks) over(order by Marks rows between unbounded preceding and unbounded following) as "Avg",
count(Marks) over(order by Marks rows between unbounded preceding and unbounded following) as "Count",
min(Marks) over(order by Marks rows between unbounded preceding and unbounded following) as "Min",
max(Marks) over(order by Marks rows between unbounded preceding and unbounded following) as "Max",
from student;
```

Window Functions(Ranking Function)

```
Select Marks,
row_number() over(order by Marks) as "row_no",
rank() over(order by Marks) as "rank",
dense_rank() over(order by Marks) as "dense_rank",
percent_rank() over(order by Marks) as "percent_rank"
from student;
```

Window Functions(Analytic Function)

```
Select Marks,
first_value(Marks) over(order by Marks) as "FirstValue",
last_value(Marks) over(order by Marks) as "LastValue",
lead(Marks) over(order by Marks) as "Lead",
lag(Marks) over(order by Marks) as "Lag"
from student;
```

Interview QA in SQL

```
# Video-1
select * from employee
# Q-1: Remove duplicate values from Employee Table?
select distinct * from employee
# Q-2: Write a Query to find out duplicate values from employee?
```

```

with cte as(select *, row_number() over(partition by employee_id order by employee_id) as rn from employee) select * from cte where rn=2
# Q-3: Write a Query to find out highest earning employee based on each position?
select max(salary), position from employee group by position
# Q-4: Write a Query to get Top 3 highest earning employee ?
select top 3 * from employee order by salary desc
with cte as(select distinct * from employee) select top 3* from cte order by salary desc
# Q-5: Write a Query to get Top 3 Lowest earning employee ?
with cte as(select distinct * from employee) select top 3* from cte order by salary asc
# Q-6: Difference b/w Union and Unionall
Union: Combines the results of two Tables and removes duplicate rows.
Unionall: Combines the results of two Tables without removing duplicates
# Q-7: Difference b/w Primary Key Vs Unique Key
PrimaryKey: Used to serve as a Unique identifier for each row in a Table. Cannot accept NULL values. Only one primary key in Table
Ex: Emp_ID, RollNumber...etc
UniqueKey: uniquely determines a row, but its not a primary key. Can accept NULL values. Table can have more than one unique key
Ex: Mobile, Email, Adhar Number...etc
# Q-8: Difference b/w Rank & dense Rank
Rank(): Skips the one Next Rank if there comes any equal Ranks Ex: Rank1, Rank2, Rank2, Rank4 # here rank3 skipped
Dense_Rank(): Doesnot skip Ranks, even if there comes equal Ranks Ex: Rank1, Rank2, Rank2, Rank3 # here No skip
# Q-9: Diff b/w Delete, Drop and Truncate
Truncate: It deletes the complete Records from the existing Table, but Schema will exists.
Drop: It drops the existing Table from the DB alongwith Schema also.
Delete: Its a DML(Data Manipulation Language) command, deletes specific records from Table in the DB. It can delete one or more Rows from the table as per our query with 'where' clause
# Q-10: What are the different Windows functions
It performs calculations across a set of Table Rows related to the current Row.
Ex: Row_Number(), Rank(), Dense_Rank(), Avg(), Over()
# Q-11: Diff b/w Where and Having clauses
Where: Filters Rows before Grouping (GroupBy)
Having: Filters groups created by GroupBy
# Q-12: How to Handle NULL values in SQL
Is NULL or Is Not Null are to filter records
Coalesce(column,default_value) to replace NULL with a new defined value Ex: unknown/others...etc words in place of NULL
# video-2
# Q-13: Write a Query to find out 2nd Highest Salary Earning Employee
with cte as(select *, row_number() over(order by salary desc) rn from emp) select * from cte where rn=2 # for 3rd highest salary rn=3
# Q-14: Write a Query to find out 2nd Lowest Salary Earning Employee
with cte as(select *, row_number() over(order by salary asc) rn from emp) select * from cte where rn=2
# Q-15: Write a Query to get 2nd Highest salary based on Each Dept
with cte as(select *, row_number() over(partition by dept order by salary desc) as rn from emp) select * from cte where rn=2
# Q-16: Write a Query to get 3rd Lowest salary based on Each Location
with cte as(select *, row_number() over(partition by location order by salary asc) as rn from emp) select * from cte where rn=3
# Q-17: Write a Query to get Bottom Two salaries based on Each Location
with cte as(select *, row_number() over(partition by location order by salary asc) as rn from emp) select * from cte where rn in (1,2)
# Q-18: Write a Query to get Top Two salaries based on Each Location
with cte as(select *, row_number() over(partition by location order by salary desc) as rn from emp) select * from cte where rn in (1,2)
# video-3
# if Two Tables having same 5 records in id column and in 2nd Table 6th id is null
# Q-19: How many rows will you get when you will perform inner Join
select * from table1 join table2 on table1.id=table2.id # Total 25 records in output
# Q-20: How many rows will you get when you will perform Left Join
select * from table1 left join table2 on table1.id=table2.id # Total 25 records in output
# Q-21: How many rows will you get when you will perform Right Join
select * from table1 right join table2 on table1.id=table2.id # Total 26 records in output
# Q-22: How many rows will you get when you will perform Full Join
select * from table1 full join table2 on table1.id=table2.id # Total 26 records in output

# if Two Tables having same 5 records in id column and 6th id is null in both Tables
# Q-23: How many rows will you get when you will perform inner Join
select * from table1 join table2 on table1.id=table2.id # Total 25 records in output, bcoz NULL won't match with other NULL
# Q-24: How many rows will you get when you will perform Left Join
select * from table1 left join table2 on table1.id=table2.id # Total 26 records in output
# Q-25: How many rows will you get when you will perform Right Join
select * from table1 right join table2 on table1.id=table2.id # Total 26 records in output
# Q-26: How many rows will you get when you will perform Full Join
select * from table1 full join table2 on table1.id=table2.id # Total 26 records in output
# video-4
# Q-1: Write query to create new table with same schema as employee table(only schema)
select * from employee ---> select * from table3
select * into table3 from employee where 1=2
# Q-2: Create new table same like employee table (data+schema)
select * into table4 from employee where 1=1
# Q-3: Write query where employee name starts with 'A'
select * from employee where employee_name like 'A%'
# Q-4: Write query where department_id starts Letter and ends Letter is same
select * from employee where left(department_id,1)=right(department_id,1)
# Q-5: Write query to get records in xml format
select * from employee for xml auto
# Q-6: How to get current date

```

```

select getdate() ---> select current_timestamp
# Q-7: Query to get current month
select month(getdate())
# Q-8: Query to get current year
select year(getdate())
# video-5
# Q-1: diff b/w union and unionall
Union: It will remove duplicates
Unionall: It combines 2 Tables and gives all records without removing duplicates
# Q-2: primary key vs unique key
Primary key: In a Table, only 1 primary key, but can have multiple unique keys. Primary key cannot be Null, but unique key may contain Null values
# Q-3: diff b/w rank and dense rank
Rank: It skips Next Rank if there tie happens Ex: 1,2,2,4
Dense_Rank: It doesnot skip, continues even tie happens Ex: 1,2,2,3
# Q-4: delete vs drop vs truncate
Truncate: It removes total of entire Table exclude schema
Delete: It deletes specific Record/Row
Drop: It deletes entire Table along with schema also.
# Q-5: what are different windows functions
These perform calculations across a set of table Rows related to the current row.
Row_number(), Rank(), Dense_Rank(), Avg(), Over()
# Q-6: diff b/w where and having clauses
Where: Filter Rows before Grouping (group by)
Having: Filter Groups After created by group by
# Q-7: how to handle Null values in SQL
Is Null or Is Not Null: To Filter Records
Coalesce: To replace Null with our defined value Ex: Unknown, Other...etc instead of Null
# video-11
# Q-1: Find out Employees' Manager details from employee table
select a.empname, b.empname as manager_name, a.department from employee a left join employee b on a.manager_id=b.manager_id
# Q-2: Write query to find out cumulative sum of salary
select *, sum(salary) over(order by employee_id) as rn from employee
# video-14
# Q-1: Fill the Null values with previous name
with cte as(select *, row_number() over(order by (select null)) rn, case when brand_name is null then 0 else 1 end rn1 from
chocolate_brands),
    cte1 as(select *, sum(rn1) over(order by rn) roll_sum from cte) select chocolate_name, brand_name, max(brand_name) over(partition
by roll_sum) as new_brand_name from cte1;
# video-15
# Q-1: Find New & Repeated visiting customers date wise
with first_visit_date as (select customer_id, min(order_date) first_order from customer_orders group by customer_id),
    final as(
    select a.*, b.first_order, case when order_date=first_order then 1 else 0 end new_customer, case when order_date!=first_order then
1 else 0 end repeated_customer
    from customer_orders a join first_visit_date b on a.customer_id=b.customer_id)
select sum(new_customer), sum(repeated_customer) as repeated , order_date from final group by order_date

```

Misc:

```

# Q-1: How to create New Table structure based on existing table structure (Note: only schema need to create)
select * into empl from employee where 1=2 # it duplicates table with schema without records
select * from empl
# Q-2: How to create new table based on existing table (Note: copy data also from one table to another)
select * into empl1 from employee where 1=1
select * from empl1
# Q-3: How to find duplicate records in a table
Method1: select count(*), id from empl1 group by id having count(*)>1
Method2: select *, row_number() over(partition by id order by id) as rn from empl1
# Q-4: find duplicate records which repeated multiple times
with cte as(
select *, row_number() over(partition by id order by id) as rn from empl1) select * from cte where rn>1
# Q-5: Remove duplicate records from a table
with cte as(
select *, row_number() over(partition by id order by id) as rn from empl1) select * from cte where rn=1
# Q-6: Find 2nd highest salary
Method1: select max(salary) from employee where salary < (select max(salary) from employee);
Method2: with cte as(select *, row_number() over(order by salary desc) as rn from employee) select * from cte where rn=2;
# Q-7: Find 3rd Highest salary
with cte as(select *, row_number() over(order by salary desc) as rn from employee) select * from cte where rn=3;
# Q-8: Find 2nd Highest salary based on each dept/post
with cte as(select *, row_number() over(partition by post order by salary desc) as rn from employee) select * from cte where rn=2
# Q-9: Find Bottom two salaried employee details
select top 2 * from employee order by salary asc
# Q-10: Find Top two salaried employee details
select top 2 * from employee order by salary desc
# Q-11: Find the Lowest salaried employee in each dept
select min(salary),post from employee group by post
# Q-12: Find 3rd to 5th salaried employees
with cte as(select *, row_number() over(order by salary desc) as rn from employee) select * from cte where rn between 3 and 5

```

Delta Tables:

Delta Lake:

Lake house:

Delta table:

Delta_log, Checkpoints, optimization

Time Travel in Delta Tables:

Vacuum in Delta Tables:

Merge in Delta Tables:

Schema Evolution in Delta Tables:

Partitioning

Liquid Clustering in Delta lake

Delta Live Tables(DLT) in databricks

Serverless & Dedicated Spark Pools

CI-CD in ADF:

Medallion Architecture:

Unity Catalog, External Location, Storage creds

Databricks secrets scope & Azure keyvault

Autoloader in databricks

Volumes in databricks

Workflows in databricks

SparkContext, SparkSession

RDD, df, dataset

Lazy Evaluation

Narrow & Wide Transformations

Databricks-Concepts:

1. Compute Cluster, Notebook, dbutils
2. Data Lake, Dataware house, Delta Lake, Delta Table, Delta Live Table(DLT), DBT
3. Unity Catalog, External Location, Storage Credentials,

PySpark-Concepts:

1. RDD, DF, Lazy Evaluation, partitioning, bucketing
2. Optimization
3. Transformations

SQL-Concepts:

1. Normal concepts
2. Joins
3. Merge, Upsert
4. Fact & Dimesion Tables
5. SCDTypes-1,2,3

Azure ADF-Concepts:

1. Datasets, Linked Services, Activity
2. Triggers
3. Managed Identity, Service Principal
4. ELT, ETL (Ingestion, Extract, Transformation, Load)

Azure Synapse-Concepts:

1. Server Pools
2. External Tables, Managed Tables
- 3.

Azure Misc-Concepts:

1. KeyVault
2. Medallion Architecture (bronze, silver, gold)
3. Star Schema